

Ciencia de Datos

APUNTES DE TEORICOS

Procesamiento, transformación, remuestreo y validación de datos

Material de apoyo para ejercicios y prácticas

Documento de consulta académica

Contenido

1. 1.1 Procesamiento de Datos
2. 1.2 Métodos de Transformación de Datos
3. 1.3 Métodos de Remuestreo
4. 1.4 Escalamiento y estandarización
5. 1.5 Normalización y Binarización
6. 1.6 Box-Cox y Yeo-Johnson
7. 1.7 Validación Cruzada
8. 1.8 División por porcentaje
9. Buenas prácticas integradoras: pipelines, fuga de información y reproducibilidad
10. Referencias en formato APA 7

Septiembre de 2026

Contenido

Presentación	4
Objetivos de aprendizaje	4
Contenido	4
1.1. Procesamiento de Datos	5
1.1.1. Concepto y propósito	5
1.1.2. Diagnóstico inicial de calidad.....	5
1.1.3. Limpieza de duplicados e inconsistencias	6
1.1.4. Valores faltantes	6
1.1.5. Datos atípicos	7
Regla del rango intercuartílico (IQR)	7
1.1.6. Evidencia antes/después	7
1.2. Métodos de Transformación de Datos.....	8
1.2.1. Qué significa transformar.....	8
1.2.2. Transformaciones monotónicas y asimetría.....	8
1.2.3. Codificación de variables categóricas	9
1.2.4. Ingeniería de variables simples	9
1.3. Métodos de Remuestreo	10
1.3.1. Idea general	10
1.3.2. Bootstrap	10
1.3.3. Clases desbalanceadas.....	10
Oversampling aleatorio.....	11
Undersampling aleatorio	11
1.3.4. Bootstrap, balanceo y validación cruzada no son lo mismo	11
1.4. Escalamiento y estandarización	12
1.4.1. Por qué importa la escala.....	12
1.4.2. Estandarización: StandardScaler	12
1.4.3. Min-Max Scaling	12
1.4.4. RobustScaler.....	12
1.4.5. ¿Cuándo es especialmente importante escalar?.....	13
1.5. Normalización y Binarización.....	14

1.5.1. Normalización por norma	14
1.5.2. Binarización	15
1.5.3. Binarización de características vs. codificación de etiquetas.....	15
1.6. Box-Cox y Yeo-Johnson	16
1.6.1. Motivación de las transformaciones de potencia.....	16
1.6.2. Transformación Box-Cox.....	16
1.6.3. Transformación Yeo-Johnson	16
1.6.4. Comparación	17
1.6.5. Qué significa “hacer más gaussiana” una variable	17
1.7. Validación Cruzada	18
1.7.1. Problema de una única partición	18
1.7.2. K-fold cross-validation	18
1.7.3. Validación cruzada estratificada	19
1.7.4. Métricas y desbalance	19
1.7.5. Validación cruzada y conjunto de prueba final	19
1.8. División por porcentaje	20
1.8.1. Entrenamiento, validación y prueba	20
1.8.2. Porcentajes comunes	20
1.8.3. Estratificación en la división	20
1.8.4. Datos temporales y grupos: excepciones importantes	20
2. Buenas prácticas integradoras	21
2.1. Fuga de información (data leakage)	21
2.2. Pipelines	21
2.3. Ajustar (fit) no es lo mismo que transformar (transform)	21
2.4. Reproducibilidad	21
2.5. Guía de decisión rápida	22
3. Glosario esencial	23
Referencias.....	24
Nota sobre las figuras	¡Error! Marcador no definido.

Presentación

La preparación de datos es una fase central del trabajo de ciencia de datos porque la calidad, la escala, la distribución y la forma en que se separan los datos condicionan los resultados de los análisis y de los modelos. Un algoritmo sofisticado no compensa automáticamente errores de captura, valores faltantes mal tratados, fuga de información o particiones de evaluación deficientes. Por ello, el preprocesamiento debe entenderse como un conjunto de decisiones estadísticas y metodológicas, no como una secuencia mecánica de funciones de software (Kuhn & Johnson, 2013; James et al., 2021).

Estos apuntes desarrollan la teoría asociada con los temas trabajados en el Notebook de clase y en las actividades prácticas para el tema: procesamiento y limpieza de datos, tratamiento de valores nulos y atípicos, transformación de variables, remuestreo, escalamiento y estandarización, normalización y binarización, transformaciones Box-Cox y Yeo-Johnson, validación cruzada y división por porcentaje. Se agrega además la noción de fuga de información y el uso conceptual de pipelines porque ambos elementos conectan correctamente el preprocesamiento con la evaluación de modelos.



Figura 1. Flujo conceptual del preprocesamiento y evaluación de datos. Fuente: elaboración propia.

Objetivos de aprendizaje

- Explicar por qué la calidad de los datos afecta la validez del análisis y del modelado.
- Distinguir valores faltantes, datos inválidos y valores atípicos, así como sus tratamientos posibles.
- Seleccionar transformaciones de acuerdo con la distribución y el significado de cada variable.
- Diferenciar escalamiento, estandarización y normalización.
- Comprender el propósito estadístico del bootstrap y el propósito predictivo del balanceo de clases.
- Explicar las condiciones de uso de Box-Cox y Yeo-Johnson.
- Aplicar el razonamiento de validación cruzada y partición entrenamiento-validación-prueba sin fuga de información.

1.1. Procesamiento de Datos

1.1.1. Concepto y propósito

El procesamiento de datos comprende las operaciones utilizadas para convertir datos crudos en una representación consistente, interpretable y adecuada para el análisis. En proyectos reales, los datos pueden contener errores de captura, unidades incompatibles, registros duplicados, categorías escritas de forma distinta, valores faltantes y observaciones extremas. El objetivo no es producir un conjunto “perfecto”, sino documentar y reducir aquellas imperfecciones que puedan sesgar la interpretación o impedir el uso correcto de métodos estadísticos y de aprendizaje automático (Kuhn & Johnson, 2013).

Una decisión de preprocesamiento debe considerar el significado de la variable y el proceso que generó los datos. Por ejemplo, una edad negativa probablemente representa un error; un ingreso extremadamente alto puede ser válido. Tratar ambos casos como si fueran equivalentes conduce a pérdida de información. La exploración gráfica y el conocimiento del dominio son, por tanto, complementarios a las reglas automáticas (Tukey, 1977; Iglewicz & Hoaglin, 1993).

1.1.2. Diagnóstico inicial de calidad

Antes de modificar un dataset conviene elaborar un diagnóstico. En una tabla estructurada deben revisarse al menos: dimensiones, tipos de variables, rangos, valores únicos, duplicados, porcentajes de datos faltantes, consistencia de categorías y estadísticas descriptivas. Esta revisión permite distinguir problemas de representación de problemas sustantivos.

Aspecto	Pregunta de diagnóstico	Ejemplo de evidencia
Estructura	¿Cuántas filas y columnas existen?	shape, info o diccionario de datos
Tipos	¿Las variables tienen el tipo esperado?	fecha almacenada como texto
Duplicados	¿Existen registros repetidos?	conteo de filas duplicadas
Faltantes	¿Qué variables tienen nulos y en qué proporción?	porcentaje de NaN por columna
Rangos	¿Hay valores imposibles o fuera de dominio?	edad = -5 o 150
Categorías	¿Las etiquetas son coherentes?	Web, web, " web "
Distribución	¿Hay asimetría o valores extremos?	histograma y boxplot

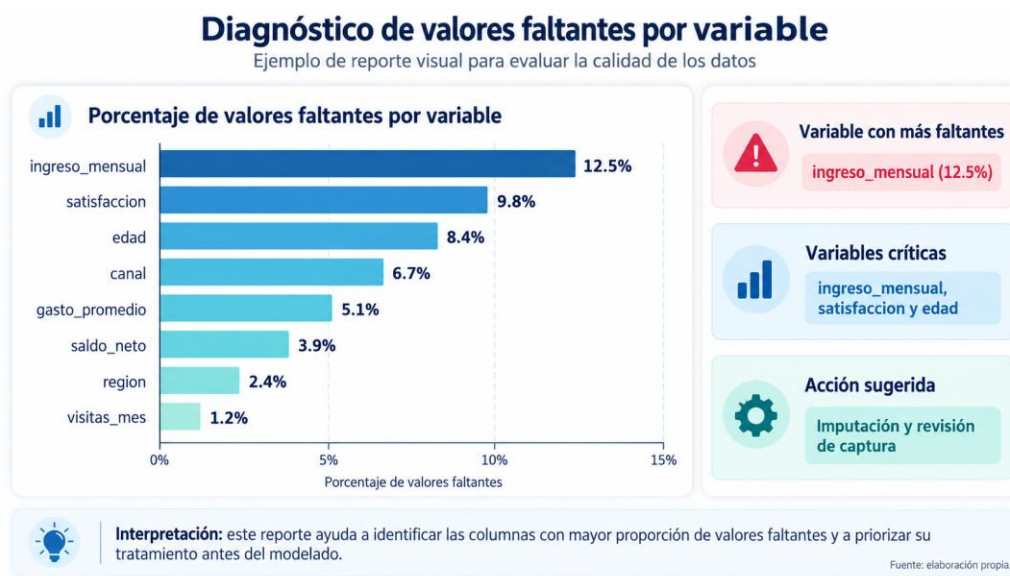


Figura 2. Ejemplo de reporte visual de porcentaje de valores faltantes

1.1.3. Limpieza de duplicados e inconsistencias

Un duplicado es una observación que aparece repetida bajo un criterio definido. La duplicidad puede ser exacta —todos los campos iguales— o lógica —el mismo cliente aparece varias veces por un error de integración—. Eliminar duplicados sin definir la unidad de análisis puede ser peligroso: dos transacciones idénticas podrían ser eventos legítimos. Por ello, antes de aplicar una función como `drop_duplicates` debe definirse qué representa una fila.

Las inconsistencias de texto se reducen mediante operaciones de estandarización: eliminación de espacios, unificación de mayúsculas/minúsculas, corrección de sinónimos y uso de catálogos. En variables numéricas debe verificarse la unidad de medida. Convertir pesos a kilogramos o monedas a una unidad común es una transformación semántica necesaria antes del análisis.

1.1.4. Valores faltantes

Los datos faltantes no constituyen un problema único. Little y Rubin (2019) distinguen mecanismos de ausencia que ayudan a razonar sobre el sesgo potencial: MCAR (Missing Completely At Random), MAR (Missing At Random) y MNAR (Missing Not At Random). En términos prácticos, la diferencia fundamental es si la probabilidad de que falte un dato está relacionada o no con información observada o no observada.

- MCAR: la ausencia no depende de variables observadas ni del valor faltante; es la situación más restrictiva.
- MAR: la ausencia puede depender de variables observadas. Por ejemplo, ciertas edades podrían tener mayor probabilidad de omitir ingreso, pero esa relación puede explicarse mediante variables conocidas.
- MNAR: la ausencia depende del propio valor no observado o de un mecanismo no explicado por las variables disponibles; por ejemplo, personas con ingresos muy altos que deciden no reportarlos precisamente por su nivel de ingreso.

La eliminación de filas con nulos es simple, pero reduce tamaño muestral y puede introducir sesgo. La imputación reemplaza valores faltantes con estimaciones. Media, mediana y moda son estrategias básicas; la mediana suele ser preferible a la media cuando hay asimetría o valores extremos. Métodos más avanzados incluyen imputación múltiple y modelos predictivos, pero su elección debe corresponder con el mecanismo de ausencia y el propósito del análisis (Little & Rubin, 2019).

Estrategia	Ventaja	Riesgo o limitación
Eliminar filas	Sencilla y transparente	Pierde información; puede sesgar si la ausencia no es aleatoria
Media	Fácil de interpretar	Sensible a atípicos y reduce artificialmente la variabilidad
Mediana	Robusta a valores extremos	No conserva relaciones multivariadas por sí sola
Moda	Útil en categorías	Puede sobrerrepresentar la categoría dominante
Imputación modelada	Aprovecha relaciones entre variables	Mayor complejidad y riesgo de fuga si se ajusta antes de particionar

Nota: En modelado predictivo, la regla correcta es aprender los parámetros de imputación únicamente con el conjunto de entrenamiento y después aplicar esos parámetros a validación y prueba.

1.1.5. Datos atípicos

Un valor atípico es una observación inusualmente distante respecto del patrón dominante. “Atípico” no significa “incorrecto”. Iglewicz y Hoaglin (1993) enfatizan que la identificación debe separarse de la decisión de eliminar o acomodar la observación. En ciencia de datos, un extremo puede representar fraude, una falla, un cliente de alto valor o un error de medición; en cada caso la acción apropiada es distinta.

Regla del rango intercuartílico (IQR)

$$IQR = Q3 - Q1$$

$$\text{Límite inferior} = Q1 - 1.5 \times IQR \quad | \quad \text{Límite superior} = Q3 + 1.5 \times IQR$$

La regla $1.5 \times IQR$ es una regla exploratoria asociada al diagrama de caja. Es robusta porque utiliza cuartiles, no media y desviación estándar. Sin embargo, no es una prueba universal de error. En distribuciones naturalmente sesgadas puede marcar muchas observaciones válidas como extremas (Tukey, 1977).

Diagrama de caja: identificación visual de valores atípicos

Cómo interpretar cuartiles, rango intercuartílico y valores extremos

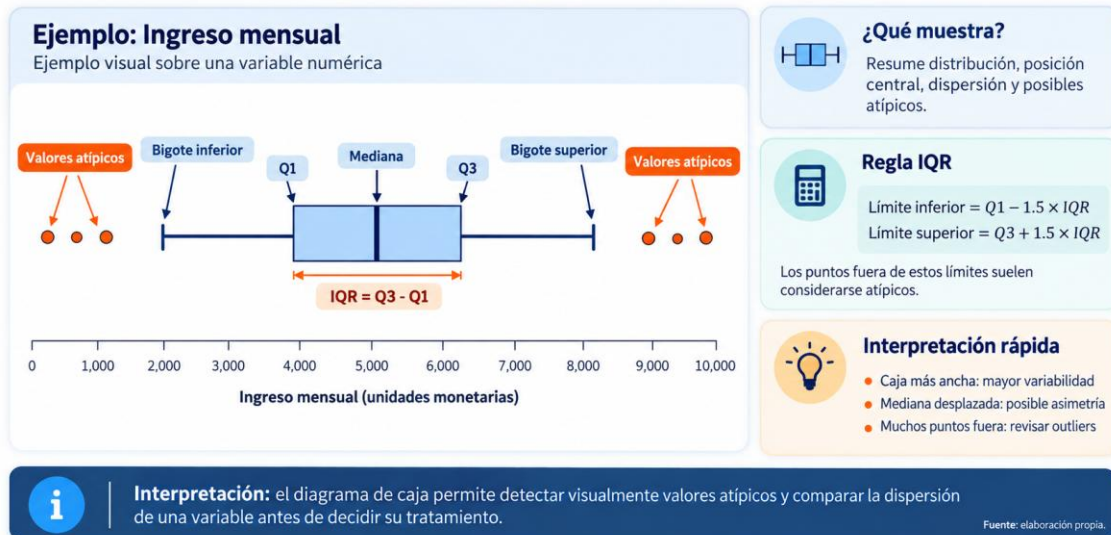


Figura 3. Diagrama de caja y observaciones potencialmente atípicas.

Tratamiento	Cuándo puede justificarse	Precaución
Conservar	El valor es real y relevante	Puede afectar métodos sensibles a escala
Corregir	Se verifica un error de captura o unidad	Debe existir evidencia del valor correcto
Eliminar	El registro es inválido o ajeno a la población	Documentar criterio y cantidad eliminada
Recortar / winsorizar	Se desea limitar influencia extrema	Modifica la distribución original
Transformar	La asimetría es estructural	La interpretación cambia de escala

1.1.6. Evidencia antes/después

Una práctica recomendable es conservar evidencia del estado antes y después de cada tratamiento. Tablas de nulos, estadísticas descriptivas, histogramas y boxplots permiten comprobar que la transformación produjo el efecto esperado. La trazabilidad también facilita explicar por qué cambió el número de registros o la distribución de una variable.

1.2. Métodos de Transformación de Datos

1.2.1. Qué significa transformar

Una transformación cambia la representación de una variable con el fin de facilitar análisis, cumplir supuestos, mejorar estabilidad numérica o convertir información no numérica en una forma que pueda utilizar un algoritmo. No todas las transformaciones pretenden “normalizar” una distribución. Algunas conservan el orden, otras crean indicadores, otras cambian unidades y otras generan nuevas características (Kuhn & Johnson, 2013).

1.2.2. Transformaciones monotónicas y asimetría

Las variables económicas, de gasto, tiempos de espera o tamaños suelen presentar asimetría positiva: muchos valores pequeños o medios y pocos valores muy grandes. En esos casos una transformación logarítmica puede comprimir la cola derecha. Para $x \geq 0$ es frecuente utilizar $\log(1 + x)$, ya que admite el valor cero y se comporta de forma similar al logaritmo para valores grandes.

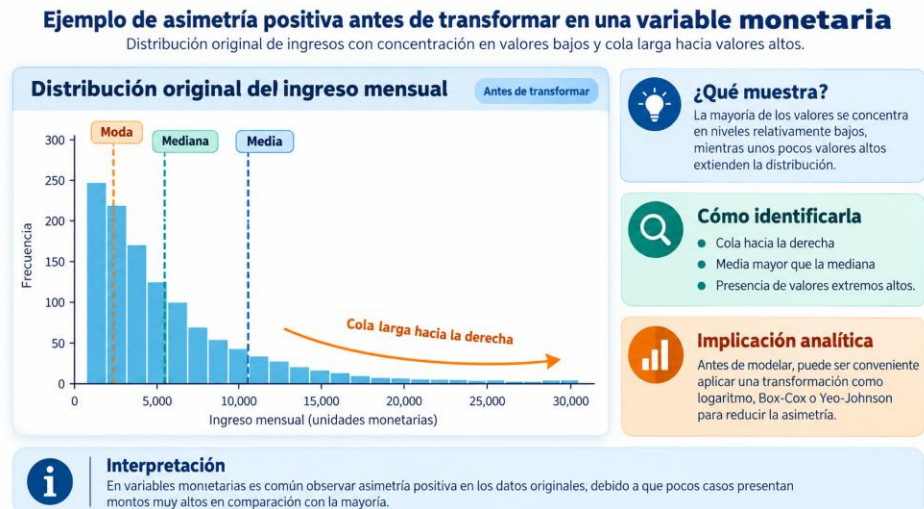


Figura 4. Ejemplo de asimetría positiva en una variable monetaria.

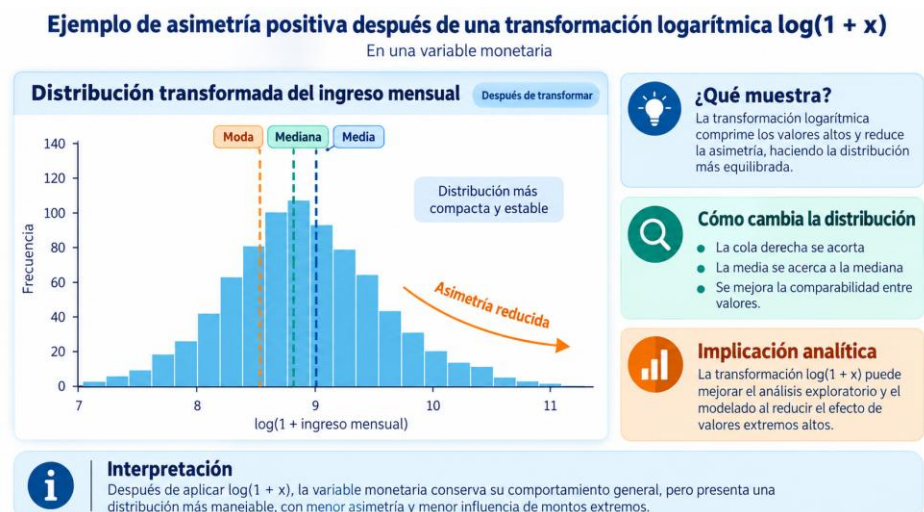


Figura 5. Efecto ilustrativo de $\log(1 + x)$ sobre la asimetría.

La transformación logarítmica cambia la interpretación de las diferencias: una diferencia aditiva en escala original se convierte en una relación multiplicativa en escala logarítmica. Por ello, debe utilizarse con una explicación del efecto que tendrá en la interpretación.

1.2.3. Codificación de variables categóricas

Una variable categórica nominal no posee una distancia numérica natural. Asignar “Web = 1”, “Tienda = 2” y “App = 3” puede inducir un orden inexistente. One-Hot Encoding crea una variable indicadora por categoría, lo que evita imponer distancias arbitrarias. Para variables ordinales sí puede existir un orden justificable, como satisfacción baja, media y alta.

Codificación	Uso típico	Consideración
One-Hot	Categorías nominales	Aumenta el número de columnas
Ordinal	Categorías con orden real	Los números deben respetar el orden conceptual
Binarización	Presencia/ausencia o umbral	Pierde detalle continuo de la variable original
Target encoding	Alta cardinalidad en modelado supervisado	Debe controlarse estrictamente la fuga de información

1.2.4. Ingeniería de variables simples

Transformar también puede significar construir variables con mayor significado analítico. Ejemplos: gasto por visita = gasto total / número de visitas; antigüedad en años = meses / 12; cliente frecuente = 1 si visitas $\geq$ umbral. Estas variables deben derivarse de una hipótesis de dominio y no solo de combinaciones arbitrarias. En un pipeline predictivo, cualquier transformación que aprenda parámetros de los datos debe ajustarse únicamente en entrenamiento.

1.3. Métodos de Remuestreo

1.3.1. Idea general

El remuestreo reutiliza los datos observados para construir nuevas muestras y estudiar la variabilidad de un estimador o modificar la composición de un conjunto de entrenamiento. Dos usos que aparecen en la práctica son conceptualmente distintos: bootstrap para inferencia y oversampling/undersampling para balanceo de clases.

1.3.2. Bootstrap

El bootstrap genera muestras del mismo tamaño que el conjunto original mediante muestreo con reemplazo. Como una observación puede seleccionarse varias veces y otra ninguna, cada réplica es ligeramente distinta. Calculando un estadístico —por ejemplo, la media— en muchas réplicas se aproxima su distribución muestral. Efron y Tibshirani (1993) sistematizaron esta metodología y sus aplicaciones.

1. Partir de una muestra observada de n casos.
2. Extraer n observaciones con reemplazo para formar una réplica bootstrap.
3. Calcular el estadístico de interés en esa réplica.
4. Repetir B veces.
5. Analizar la distribución de los B estadísticos: media, error estándar o percentiles.

Réplica bootstrap: $\{x^*1, x^*2, \dots, x^*n\}$ extraída con reemplazo de $\{x1, x2, \dots, xn\}$

El bootstrap no “crea información nueva”; reorganiza la muestra observada para estudiar cuánto podría variar un resultado bajo un mecanismo de remuestreo. Es especialmente útil cuando la distribución teórica de un estimador es difícil de obtener.

1.3.3. Clases desbalanceadas

En clasificación, un conjunto está desbalanceado cuando una o más clases aparecen en proporciones muy diferentes. Esto puede ser crítico cuando la clase minoritaria representa el evento de mayor interés —fraude, falla, abandono o enfermedad—. Los algoritmos estándar pueden producir una accuracy alta prediciendo principalmente la clase mayoritaria, por lo que la evaluación debe considerar métricas como recall, precisión, F1 o AUC según el problema (He & Garcia, 2009).

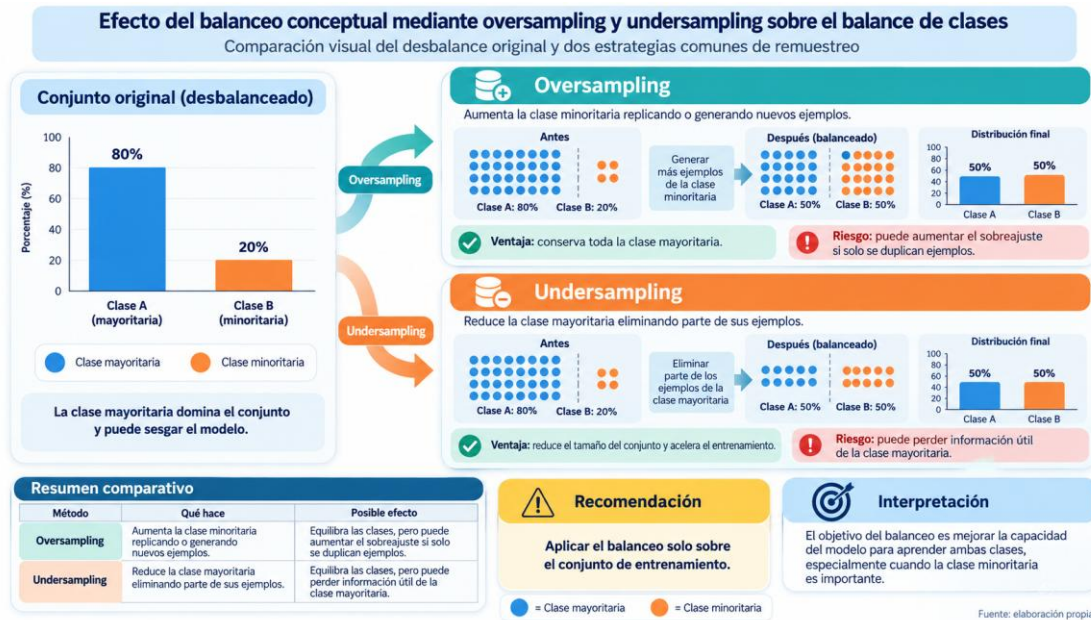


Figura 6. Balanceo conceptual mediante oversampling y undersampling.

Oversampling aleatorio

El oversampling replica observaciones de la clase minoritaria hasta aumentar su representación. Su ventaja es que conserva los ejemplos mayoritarios; su desventaja es que repetir los mismos casos puede favorecer sobreajuste. Métodos como SMOTE generan ejemplos sintéticos a partir de vecinos de la clase minoritaria, pero su uso también requiere validación cuidadosa (Chawla et al., 2002).

Undersampling aleatorio

El undersampling reduce la clase mayoritaria. Disminuye costo computacional y equilibra la muestra, pero puede descartar información útil. Es más razonable cuando la clase mayoritaria es muy grande y redundante.

Nota: El balanceo debe aplicarse únicamente sobre el conjunto de entrenamiento. Si se remuestrea antes de separar prueba, copias o derivados de una misma observación pueden aparecer tanto en entrenamiento como en prueba y producir una evaluación optimista.

1.3.4. Bootstrap, balanceo y validación cruzada no son lo mismo

Método	Objetivo	¿Con reemplazo?	Resultado principal
Bootstrap	Estimar variabilidad/incertidumbre	Generalmente sí	Distribución de un estadístico
Oversampling	Aumentar clase minoritaria	Puede ser sí o sintético	Entrenamiento más balanceado
Undersampling	Reducir clase mayoritaria	No	Entrenamiento más balanceado
Validación cruzada	Estimar rendimiento de generalización	No es su principio central	Métricas sobre múltiples particiones

1.4. Escalamiento y estandarización

1.4.1. Por qué importa la escala

Las variables pueden expresarse en magnitudes muy diferentes: edad en decenas, satisfacción en una escala 1–10 e ingreso en miles. En métodos basados en distancias, gradientes o penalizaciones, una variable de gran magnitud puede dominar numéricamente a otras aunque no sea conceptualmente más importante. El preprocesamiento por escala busca colocar variables en representaciones comparables (Kuhn & Johnson, 2013; Hastie et al., 2009).

1.4.2. Estandarización: StandardScaler

$$z = (x - \mu) / \sigma$$

La estandarización resta la media μ y divide entre la desviación estándar σ calculadas en los datos de entrenamiento. Como resultado, la variable transformada tiene media cercana a 0 y desviación estándar cercana a 1 en el conjunto usado para ajustar el escalador. La estandarización no garantiza una distribución normal; solo centra y reescala (scikit-learn developers, n.d.-a).

La frase “estandarizar hace normal una variable” es incorrecta. Una distribución fuertemente sesgada seguirá sesgada, aunque se exprese en unidades de desviación estándar.

1.4.3. Min-Max Scaling

$$x' = (x - x_{\min}) / (x_{\max} - x_{\min})$$

Min-Max Scaling lleva normalmente la variable al intervalo [0, 1]. Conserva el orden y las proporciones lineales respecto del rango, pero depende fuertemente de los extremos observados. Un solo valor muy grande puede comprimir el resto de la distribución cerca de cero.

1.4.4. RobustScaler

RobustScaler utiliza estadísticas robustas, típicamente la mediana y el rango intercuartílico, para centrar y escalar. Por ello, los parámetros de la transformación están menos influenciados por unos pocos valores extremos que la media y la desviación estándar. Esto no elimina los outliers; solamente reduce su influencia sobre la estimación de escala (scikit-learn developers, n.d.-b).

Comparación ilustrativa de escala original, estandarizada y Min-Max

La misma variable bajo distintas escalas en un solo plano comparativo

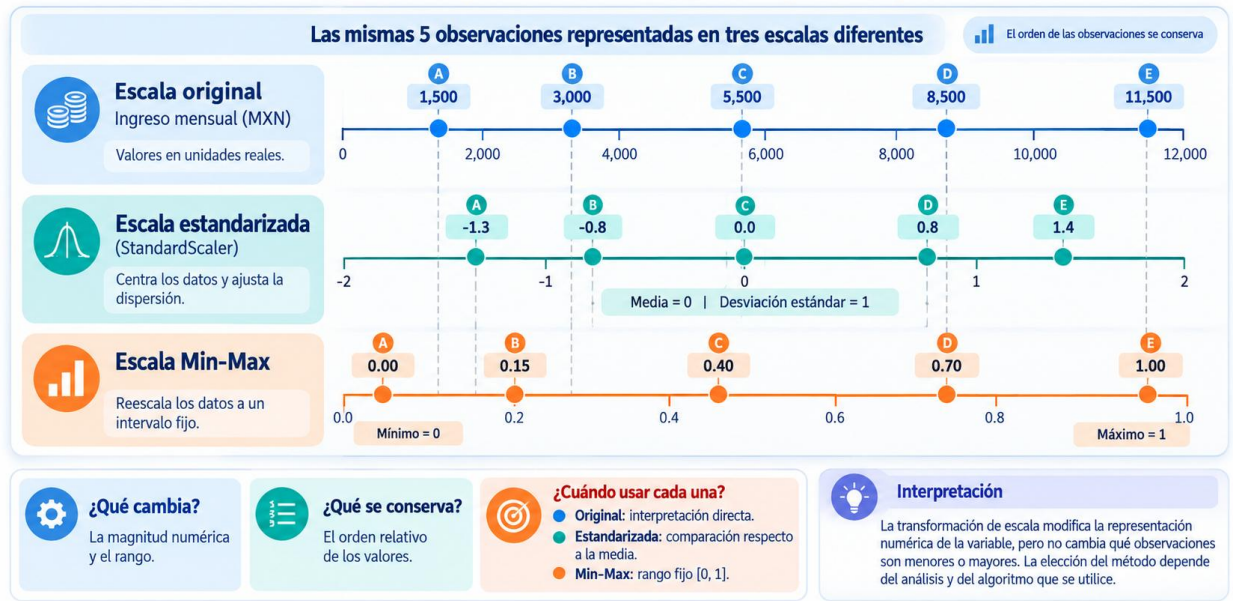


Figura 7. Comparación ilustrativa de escala original, estandarizada y Min-Max.

Método	Centro/Escala	Rango resultante	Sensibilidad a atípicos
StandardScaler	Media y desviación estándar	No acotado	Alta/moderada
MinMaxScaler	Mínimo y máximo	Usualmente [0,1]	Alta
RobustScaler	Mediana e IQR	No acotado	Menor

1.4.5. ¿Cuándo es especialmente importante escalar?

- K-nearest neighbors y clustering basados en distancia.
- Máquinas de soporte vectorial con kernels que dependen de distancias.
- Regresión logística y modelos lineales con regularización L1/L2.
- Redes neuronales y optimización por gradiente.
- PCA y otros métodos donde la varianza relativa influye en los componentes.

En árboles de decisión y bosques aleatorios la escala suele ser mucho menos relevante porque las divisiones se basan en umbrales por variable. Aun así, un pipeline puede mantener consistencia si se comparan modelos heterogéneos.

1.5. Normalización y Binarización

1.5.1. Normalización por normalización

En scikit-learn, Normalizer actúa sobre cada observación o vector, no sobre cada columna. Esto lo distingue de StandardScaler y MinMaxScaler. Con normalización L2, cada fila se divide entre su longitud euclidiana, de modo que su norma final es 1. Esta transformación es útil cuando la dirección del vector importa más que su magnitud, como en representaciones de documentos y similitud coseno (scikit-learn developers, n.d.-a).

$$\|x\|_2 = \sqrt{(x_1^2 + x_2^2 + \dots + x_p^2)} \quad \text{y} \quad x' = x / \|x\|_2$$

Normalización L2: se conserva la dirección y cambia la magnitud

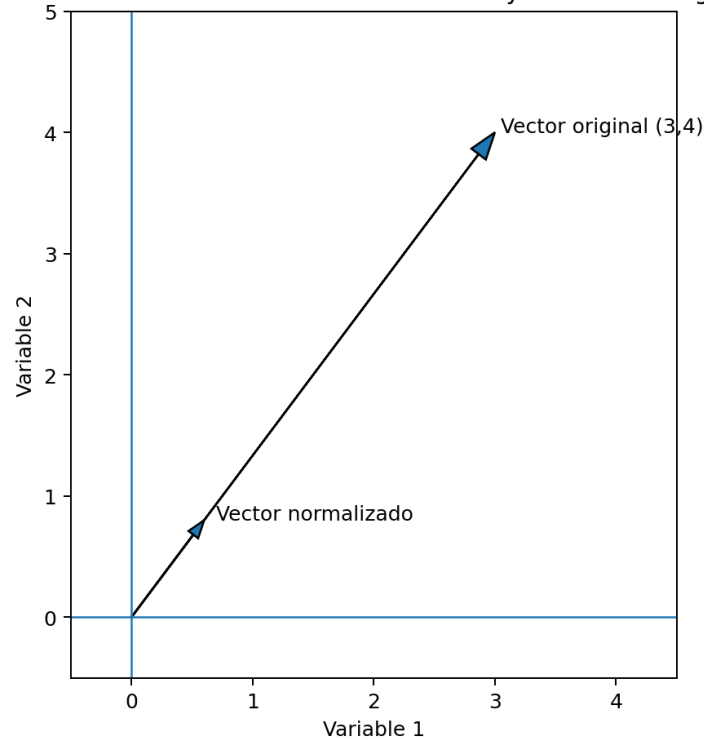


Figura 8. Efecto conceptual de la normalización L2 sobre un vector.

Concepto	Opera principalmente sobre	Objetivo
Estandarización	Columnas/características	Media 0 y escala comparable
Min-Max	Columnas/características	Rango acotado
Normalización L1/L2	Filas/observaciones	Norma unitaria por vector

1.5.2. Binarización

La binarización transforma una variable numérica en 0 o 1 de acuerdo con un umbral. Si t es el umbral, una regla típica es $x' = 1$ cuando $x > t$ y 0 en caso contrario. El procedimiento simplifica la representación y puede corresponder con una decisión de negocio —por ejemplo, satisfacción alta si puntuación > 7 —. Sin embargo, reduce información porque diferencias entre valores que quedan del mismo lado del umbral desaparecen.

$$x' = 1 \text{ si } x > t; \quad x' = 0 \text{ en caso contrario}$$

El umbral debe justificarse. Puede provenir de una regla de negocio, una norma técnica, una mediana, un percentil o un criterio optimizado dentro del entrenamiento. Si el umbral se elige observando etiquetas de todo el dataset, también puede producir fuga de información.

1.5.3. Binarización de características vs. codificación de etiquetas

Binarizer se utiliza sobre características numéricas. LabelBinarizer o una representación one-vs-rest se emplean para etiquetas de clasificación. Aunque ambos producen ceros y unos, su significado estadístico no es el mismo: uno transforma una entrada; el otro representa una variable objetivo categórica.

1.6. Box-Cox y Yeo-Johnson

1.6.1. Motivación de las transformaciones de potencia

Las transformaciones de potencia buscan encontrar una escala donde la relación entre variables sea más simple, la varianza sea más estable y/o la distribución sea más simétrica. Box y Cox (1964) propusieron una familia parametrizada por λ para seleccionar una transformación adecuada, comúnmente mediante máxima verosimilitud.

1.6.2. Transformación Box-Cox

Para $y > 0$: $y(\lambda) = (y^\lambda - 1) / \lambda$, si $\lambda \neq 0$; $y(0) = \ln(y)$

Box-Cox requiere datos estrictamente positivos. Valores cero o negativos violan su dominio en la formulación habitual. El parámetro λ determina la forma de la transformación: $\lambda = 1$ se aproxima a una transformación lineal, $\lambda = 0$ al logaritmo y otros valores producen diferentes grados de compresión o expansión (Box & Cox, 1964).

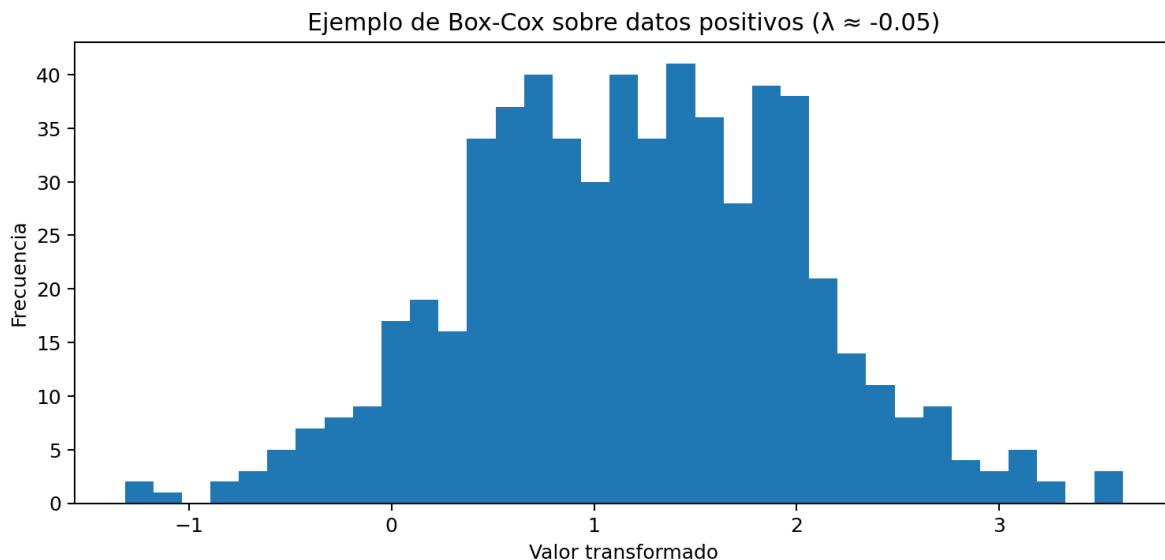


Figura 9. Distribución ilustrativa después de aplicar Box-Cox a datos positivos.

1.6.3. Transformación Yeo-Johnson

Yeo y Johnson (2000) propusieron una familia similar que está definida en toda la recta real. Esta propiedad permite trabajar con valores negativos, cero y positivos sin desplazar artificialmente toda la variable. Por ello, es una alternativa práctica cuando Box-Cox no puede aplicarse directamente.

La transformación Yeo-Johnson utiliza expresiones distintas para $y \geq 0$ y $y < 0$, manteniendo continuidad y una parametrización mediante λ . Al igual que Box-Cox, el parámetro puede estimarse para mejorar simetría o aproximación a normalidad (Yeo & Johnson, 2000).

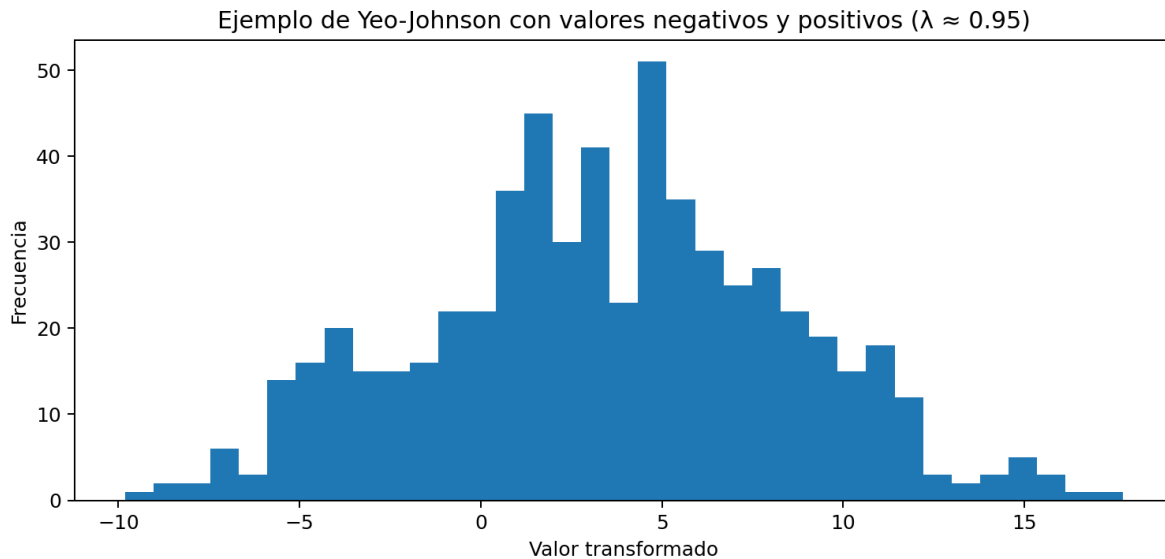


Figura 10. Ejemplo de Yeo-Johnson cuando existen valores negativos y positivos.

1.6.4. Comparación

Criterio	Box-Cox	Yeo-Johnson
Dominio	Solo valores > 0	Valores negativos, cero y positivos
Parámetro	λ	λ
Objetivo típico	Reducir asimetría / estabilizar varianza	Reducir asimetría / estabilizar varianza
Uso práctico	Variables positivas: ingresos, tiempos, concentraciones	Variables con signo: balances, cambios, residuales
Estandarización posterior	Opcional; algunas implementaciones la incluyen	Opcional; algunas implementaciones la incluyen

1.6.5. Qué significa “hacer más gaussiana” una variable

Una transformación de potencia no garantiza normalidad perfecta ni convierte automáticamente un problema en uno correcto. El objetivo puede ser reducir sesgo, estabilizar la varianza o mejorar el comportamiento de un método. Después de transformar, deben revisarse histogramas, cuantiles y el efecto sobre la interpretación y el rendimiento. La selección no debe basarse únicamente en que una gráfica “se vea normal”.

En PowerTransformer de scikit-learn, Box-Cox y Yeo-Johnson se aplican por característica y el parámetro óptimo se estima por máxima verosimilitud; Box-Cox exige valores estrictamente positivos, mientras que Yeo-Johnson admite valores con signo (scikit-learn developers, n.d.-b).

1.7. Validación Cruzada

1.7.1. Problema de una única partición

Cuando un modelo se evalúa con una única división entrenamiento/prueba, la métrica obtenida depende de qué observaciones quedaron en cada subconjunto. Si la muestra es moderada, una partición afortunada o desfavorable puede cambiar sustancialmente el resultado. La validación cruzada reduce esta dependencia al repetir entrenamiento y evaluación sobre múltiples particiones (Hastie et al., 2009; James et al., 2021).

1.7.2. K-fold cross-validation

1. Dividir los datos de desarrollo en K bloques aproximadamente iguales.
2. Entrenar con K – 1 bloques.
3. Evaluar con el bloque restante.
4. Repetir hasta que cada bloque haya funcionado una vez como validación.
5. Promediar las métricas y observar su variabilidad.

$$CV = (1 / K) \times \sum \text{métrica}_k$$

Esquema de validación cruzada k-fold: cada bloque se usa una vez para validar



Figura 11. Esquema de validación cruzada con K = 5.

Kohavi (1995) comparó variantes de validación cruzada y bootstrap para estimar accuracy y selección de modelos, y reportó buen comportamiento de la validación cruzada estratificada de 10 folds en los conjuntos estudiados. En la práctica moderna son comunes K = 5 o K = 10, pero la elección depende del tamaño de muestra, costo computacional y objetivo de evaluación.

1.7.3. Validación cruzada estratificada

En clasificación, StratifiedKFold busca preservar aproximadamente la proporción de cada clase en cada fold. Esto es especialmente útil cuando una clase es minoritaria, ya que reduce el riesgo de obtener folds con muy pocos ejemplos positivos. La estratificación no resuelve por sí misma el problema de clases desbalanceadas, pero estabiliza la composición de las particiones (scikit-learn developers, n.d.-c).

1.7.4. Métricas y desbalance

Accuracy es la proporción de predicciones correctas, pero puede ser engañosa. Si 95 % de casos pertenecen a la clase 0, un clasificador que siempre prediga 0 alcanza 95 % de accuracy sin detectar ningún caso de la clase 1. En problemas desbalanceados debe seleccionarse una métrica coherente con el costo del error: recall para priorizar detección, precision para limitar falsos positivos, F1 para balancear precision/recall, o AUC cuando interesa la capacidad de discriminación a diferentes umbrales (He & Garcia, 2009).

1.7.5. Validación cruzada y conjunto de prueba final

La validación cruzada se utiliza normalmente dentro del conjunto de desarrollo para comparar configuraciones, elegir hiperparámetros o estimar variabilidad. Si se usa el conjunto de prueba repetidamente para tomar decisiones, deja de ser una evaluación independiente. Por ello, una práctica sólida mantiene un test final sin tocar hasta que el proceso de selección ha concluido (Hastie et al., 2009).

1.8. División por porcentaje

1.8.1. Entrenamiento, validación y prueba

La partición por porcentaje asigna observaciones a subconjuntos con funciones distintas. El entrenamiento se usa para ajustar parámetros; la validación ayuda a seleccionar modelos o configuraciones; la prueba proporciona una estimación final sobre datos no utilizados en decisiones previas.

Subconjunto	Función	¿Puede influir en decisiones del modelo?
Entrenamiento	Ajustar parámetros y transformadores	Sí
Validación	Comparar alternativas e hiperparámetros	Sí
Prueba	Evaluación final de generalización	No; debe reservarse para la evaluación final

1.8.2. Porcentajes comunes

No existe un porcentaje universal. Divisiones como 80/20 o 70/30 son habituales cuando se requiere entrenamiento y prueba. Cuando se necesita un conjunto de validación independiente pueden usarse 60/20/20, 70/15/15 u otras proporciones. La decisión depende del tamaño total: con millones de registros, incluso 5 % puede ser un test enorme; con pocos cientos, separar demasiado reduce la capacidad de entrenar.

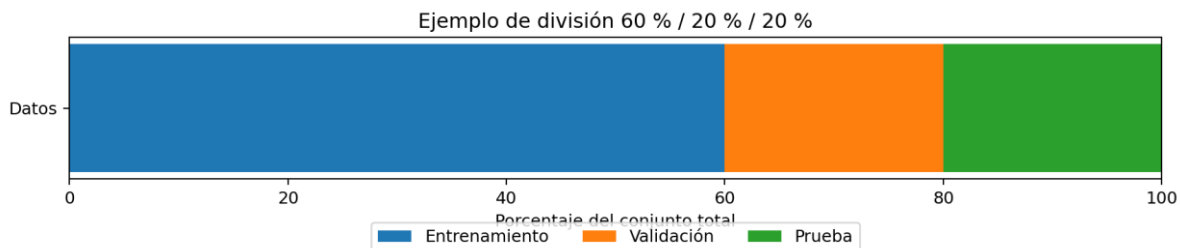


Figura 12. Representación de una división 60 % entrenamiento, 20 % validación y 20 % prueba.

1.8.3. Estratificación en la división

En clasificación, stratify=y procura mantener proporciones de clase similares entre entrenamiento y prueba. Esto reduce variaciones accidentales en la distribución de la variable objetivo y es particularmente importante con clases poco frecuentes.

1.8.4. Datos temporales y grupos: excepciones importantes

Una división aleatoria no es correcta en todos los problemas. Para series temporales debe respetarse el orden temporal: el futuro no debe utilizarse para predecir el pasado. Si existen múltiples registros por persona, máquina o empresa, también puede ser necesario separar por grupo para evitar que información muy relacionada aparezca en ambos lados. La regla general es que el esquema de evaluación debe imitar el modo en que el modelo recibirá datos nuevos en producción.

2. Buenas prácticas integradoras

2.1. Fuga de información (data leakage)

Existe fuga de información cuando el proceso de entrenamiento utiliza datos que no deberían estar disponibles en el momento real de predicción o cuando información del conjunto de validación/prueba influye en el ajuste. La fuga puede producir métricas aparentemente excelentes que no se reproducen en datos verdaderamente nuevos.

- Calcular la media global antes de separar y usarla para imputar entrenamiento y prueba.
- Ajustar StandardScaler con todos los datos y luego evaluar.
- Seleccionar variables examinando directamente el desempeño del test.
- Aplicar oversampling antes de separar entrenamiento y prueba.
- Crear una característica que utiliza información ocurrida después del evento que se pretende predecir.

2.2. Pipelines

Un pipeline encadena transformaciones y modelo de forma que cada operación se ajuste únicamente con los datos de entrenamiento correspondientes. Durante validación cruzada, el pipeline vuelve a aprender imputadores, escaladores y codificadores en cada fold. Esta estructura reduce errores de implementación y ayuda a evitar fuga de información. Scikit-learn fue diseñado con una API consistente para integrar estos componentes en flujos reproducibles (Pedregosa et al., 2011).

2.3. Ajustar (fit) no es lo mismo que transformar (transform)

Operación	Qué hace	Ejemplo
fit	Aprende parámetros a partir de datos	Media y desviación estándar de entrenamiento
transform	Aplica parámetros ya aprendidos	Usar esas mismas μ y σ en validación
fit_transform	Aprende y transforma el mismo conjunto	Conveniente dentro del entrenamiento

La separación conceptual fit/transform es fundamental para comprender por qué no deben recalcularse estadísticas de escala sobre el conjunto de prueba.

2.4. Reproducibilidad

Una práctica de ciencia de datos debe poder repetirse. Cuando existe aleatoriedad —particiones, bootstrap, remuestreo o inicialización de modelos— conviene fijar una semilla y documentarla. Sin embargo, una sola semilla no sustituye la validación: la reproducibilidad permite repetir el experimento; la validación evalúa si la conclusión es estable.

- Registrar la versión del dataset o la fuente.
- Mantener el código de limpieza y transformación en funciones o pipelines.
- Fijar random_state cuando proceda.
- Conservar parámetros, umbrales y reglas de negocio.
- Guardar métricas de cada fold y no solo el promedio.
- Documentar qué transformaciones se aplican y por qué.

2.5. Guía de decisión rápida

Situación observada	Pregunta	Técnica candidata
Valores faltantes	¿Por qué faltan y cuánto representan?	Eliminación o imputación justificada
Valores extremos	¿Son errores o casos reales?	Conservar, recortar, transformar o corregir
Asimetría positiva	¿La escala multiplicativa tiene sentido?	$\log_{10}$, Box-Cox o Yeo-Johnson
Escalas muy diferentes	¿El algoritmo depende de distancias/gradientes?	StandardScaler, MinMax o RobustScaler
Vectores cuya dirección importa	¿La magnitud total debe ignorarse?	Normalizer L1/L2
Necesidad de indicador 0/1	¿Existe un umbral defendible?	Binarizer o regla binaria
Clases desbalanceadas	¿La clase minoritaria es crítica?	Métricas adecuadas y remuestreo solo en train
Evaluación inestable	¿Una única partición es insuficiente?	Validación cruzada
Evaluación final	¿El test ha permanecido independiente?	Train/val/test o train/test bien separados

3. Glosario

Término	Definición
Asimetría	Falta de simetría de una distribución respecto de su centro.
Bootstrap	Remuestreo con reemplazo utilizado para aproximar la distribución de un estadístico.
Dato atípico	Observación inusualmente distante del patrón dominante; no implica necesariamente error.
Estandarización	Transformación que centra y reescala una variable, normalmente a media 0 y desviación estándar 1.
Estratificación	Partición que intenta conservar la proporción de clases de la variable objetivo.
Fuga de información	Uso, durante el ajuste o selección, de información que no debería estar disponible.
IQR	Rango intercuartílico: $Q3 - Q1$.
Imputación	Sustitución de valores faltantes por estimaciones.
Normalización L2	Reescalamiento de cada vector para que su norma euclidiana sea 1.
Oversampling	Aumento de la representación de la clase minoritaria.
Pipeline	Secuencia reproducible de transformaciones y modelado.
Transformación de potencia	Familia de transformaciones parametrizadas, como Box-Cox o Yeo-Johnson.
Undersampling	Reducción de la representación de la clase mayoritaria.
Validación cruzada	Evaluación repetida del modelo en múltiples particiones de entrenamiento/validación.

3. Declaraciones.

Para la elaboración de algunos contenidos de este material didáctico se utilizaron herramientas de Inteligencia Artificial generativa como apoyo en la creación de algunas imágenes, recursos visuales y contenidos explicativos. Su utilización tuvo exclusivamente fines académicos, demostrativos y de apoyo a la comprensión de los temas abordados.

Los contenidos generados o asistidos mediante Inteligencia Artificial fueron revisados, seleccionados, adaptados y validados por el docente antes de su incorporación al documento. La responsabilidad sobre la pertinencia, precisión y uso académico del material corresponde al autor de estos apuntes.

Esta declaración se incluye también como referencia para promover entre los estudiantes un uso transparente, responsable y ético de las herramientas de Inteligencia Artificial en sus trabajos académicos, recomendando identificar su utilización cuando estas herramientas hayan contribuido de manera significativa a la generación, modificación o presentación de contenidos.

Referencias

- Box, G. E. P., & Cox, D. R. (1964). An analysis of transformations. *Journal of the Royal Statistical Society: Series B (Methodological)*, 26(2), 211–243. <https://doi.org/10.1111/j.2517-6161.1964.tb00553.x>
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357. <https://doi.org/10.1613/jair.953>
- Efron, B., & Tibshirani, R. J. (1993). *An introduction to the bootstrap*. Chapman & Hall. ISBN 978-0-412-04231-7.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer. <https://doi.org/10.1007/978-0-387-84858-7>
- He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284. <https://doi.org/10.1109/TKDE.2008.239>
- Iglewicz, B., & Hoaglin, D. C. (1993). *How to detect and handle outliers*. ASQC Quality Press. ISBN 978-0-87389-247-6.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning: With applications in R* (2nd ed.). Springer. <https://doi.org/10.1007/978-1-0716-1418-1>
- Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)* (pp. 1137–1145).
- Kuhn, M., & Johnson, K. (2013). *Applied predictive modeling*. Springer. <https://doi.org/10.1007/978-1-4614-6849-3>
- Little, R. J. A., & Rubin, D. B. (2019). *Statistical analysis with missing data* (3rd ed.). Wiley. <https://doi.org/10.1002/9781119482260>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., VanderPlas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Scikit-learn developers. (n.d.-a). Preprocessing data. Scikit-learn. Retrieved September 4, 2026, from <https://scikit-learn.org/stable/modules/preprocessing.html>
- Scikit-learn developers. (n.d.-b). Compare the effect of different scalers on data with outliers. Scikit-learn. Retrieved September 4, 2026, from https://scikit-learn.org/stable/auto_examples/preprocessing/plot_all_scaling.html
- Scikit-learn developers. (n.d.-c). StratifiedKFold. Scikit-learn. Retrieved September 4, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html
- Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley. ISBN 978-0-201-07616-5.
- Yeo, I.-K., & Johnson, R. A. (2000). A new family of power transformations to improve normality or symmetry. *Biometrika*, 87(4), 954–959. <https://doi.org/10.1093/biomet/87.4.954>